Operating Systems Lab Project Report

Project 2 – Advanced Scheduling Algorithms

Team DetailsTeam Contributions

S.No	Member Name	Admission Number	Contribution
1	Mohammad Umar Shaikh Mohd Abdul Sattar	21JE0561	Implemented Multilevel Feedback Queue (MLFQ) Scheduling Algorithm and overall project integration
2	Raja Babu	21JE0740	Implemented Lottery Scheduling Algorithm
3	Tak Abhishek Anil	21JE0979	Implemented Earliest Deadline First (EDF) Scheduling Algorithm
4	Udit Singh Chauhan	21JE1002	Implemented Rate Monotonic Scheduling (RMS) Algorithm

---

1. Introduction

This project focuses on the implementation and analysis of advanced CPU scheduling algorithms used in operating systems, especially for multiprocessor systems. Scheduling is an important concept that decides how processes are executed efficiently.

In this project, we implemented multiple scheduling algorithms and compared their performance using different metrics like waiting time, turnaround time, and CPU utilization.

---

2. Objectives

- To understand how scheduling algorithms work
- To implement different scheduling techniques using C
- To simulate process execution
- To compare performance of different algorithms
- To visualize scheduling using Gantt charts

---

3. Scheduling Algorithms Implemented

3.1 Multilevel Feedback Queue (MLFQ)

- Uses multiple queues with different priorities
- Processes can move between queues
- Short jobs get higher priority
- Prevents starvation using aging

3.2 Lottery Scheduling

- Each process is assigned lottery tickets
- CPU is given randomly based on ticket draw
- More tickets → higher chance of execution
- Simple and probabilistic scheduling

3.3 Earliest Deadline First (EDF)

- Real-time scheduling algorithm
- Process with earliest deadline is executed first
- Useful in time-critical systems

3.4 Rate Monotonic Scheduling (RMS)

- Fixed priority real-time scheduling algorithm
- Priority is assigned based on process period
- Shorter period → higher priority
- Works well for periodic tasks
- Easy to implement but less flexible than EDF

---

4. System Design

- Language Used: C++
- Data Structures: Arrays for storing process details

Inputs:

- Process ID
- Arrival Time
- Burst Time
- Priority / Deadline / Tickets

Outputs:

- Execution Order
- Gantt Chart
- Performance Metrics

---

5. Performance Metrics

- **Turnaround Time (TAT)** = Completion Time – Arrival Time
- **Waiting Time (WT)** = Turnaround Time – Burst Time
- **Response Time** = First execution – Arrival Time
- **CPU Utilization** = (CPU busy time / total time) × 100

---

6. Implementation Details

- Each algorithm is implemented as a separate function
- User can choose algorithm from menu
- Input is taken from user at runtime
- Scheduling is simulated step-by-step
- Output is displayed in tabular form

---

7. Sample Output

- Process Table
- Gantt Chart
- Calculated Metrics

```
[singhcu@Udits-MacBook-Air OS % g++ Proj_2.cpp -o 21je1002_mlfq
[singhcu@Udits-MacBook-Air OS % ./21je1002_mlfq
Enter the number of queues:
3
Enter time quantum for Queue 0:
2
Enter time quantum for Queue 1:
4
Enter time quantum for Queue 2 (Enter -1 for FCFS):
-1
Enter the number of processes:
4
Enter ID, Arrival Time, and Burst Time for Process 1 (separated by spaces):
P1 0 10
Enter ID, Arrival Time, and Burst Time for Process 2 (separated by spaces):
P2 1 4
Enter ID, Arrival Time, and Burst Time for Process 3 (separated by spaces):
P3 2 7
Enter ID, Arrival Time, and Burst Time for Process 4 (separated by spaces):
P4 4 2

Starting MLFQ Simulation...

Time    CPU Running
-------------------
0       P1
1       P1
2       P2
3       P2
4       P3
5       P3
6       P4
7       P4
8       P1
9       P1
10      P1
11      P1
12      P2
13      P2
14      P3
15      P3
16      P3
17      P3
18      P1
19      P1
20      P1
21      P1
22      P3

Performance Metrics:
ID      Arrival     Burst     Completion    Turnaround    Waiting     Response
-------------------------------------------------------------------------------
P1      0           10        22            22            12          0
P2      1           4         14            13            9           1
P3      2           7         23            21            14          2
P4      4           2         8             4             2           2
-------------------------------------------------------------------------------
Average Turnaround Time : 15.00
Average Waiting Time    : 9.25
Average Response Time   : 1.25
```

```
PS D:\OS_Projects\Project_2> g++ lottery.cpp -o lottery
PS D:\OS_Projects\Project_2> ./lottery
Enter Number of Processes: 3
Enter Time Quantum: 2
Enter ID, Arrival, Burst, and Tickets for P1: P1
0
6
70
Enter ID, Arrival, Burst, and Tickets for P2: P2
0
4
20
Enter ID, Arrival, Burst, and Tickets for P3: P3
0
2
10

Visual Gantt Chart:
|P2|P2|P1|P1|P1|P1|P1|P1|P3|P3|P2|P2|

ID      Arrival Burst   Tickets Exit    Turnaround      Waiting
P1      0       6       70      8       8               2
P2      0       4       20      12      12              8
P3      0       2       10      10      10              8

Average Waiting Time: 6.00
Average Turnaround Time: 10.00
PS D:\OS_Projects\Project_2>
```

```
Enter number of processors: 1
Enter number of processes: 3
Enter details (Process_ID Arrival_Time Burst_Time Deadline):
P1 0 3 10
P2 0 6 12
P3 0 12 5

P       AT      BT      IO      DL      CT      TAT     WT      Miss
P1      0       3       0       10      15      15      12      Yes
P2      0       6       0       12      21      21      15      Yes
P3      0       12      0       5       12      12      0       Yes

Metrics:
Avg TAT: 16.00
Avg WT: 9.00
Deadline Misses: 3

CPU 0 Exec Graph:
|  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P3  |  P1  |  P1  |  P1  |  P2  |  P2  |  P2  |  P2
  |  P2  |  P2  |
0       1       2       3       4       5       6       7       8       9       10      11      12      13      14      15      16      17      18
 19      20      21
```

```
rajababu@Rajas-MacBook-Air OS_LAB % g++ rms_21je0740.cpp -o rms_21je074
rajababu@Rajas-MacBook-Air OS_LAB % ./rms_21je0740
Enter number of processes: 3
Enter Process_ID Arrival_Time Burst_Time Period/Deadline:
P1 0 3 10
P2 0 2 5
P3 0 1 4

--- Starting RMS Simulation for Hyperperiod: 20 ---

Execution Log (CPU Timeline):
Time            Process in CPU
------------------------------
[0 -> 1]        P3
[1 -> 2]        P2
[2 -> 3]        P2
[3 -> 4]        P1
[4 -> 5]        P3
[5 -> 6]        P2
[6 -> 7]        P2
[7 -> 8]        P1
[8 -> 9]        P3
[9 -> 10]       P1
[10 -> 11]      P2
[11 -> 12]      P2
[12 -> 13]      P3
[13 -> 14]      P1
[14 -> 15]      P1
[15 -> 16]      P2
[16 -> 17]      P3
[17 -> 18]      P2
[18 -> 19]      P1
[19 -> 20]      IDLE

--- Performance & Schedulability Report ---
Process P1 Missed Deadlines: 0
Process P2 Missed Deadlines: 0
Process P3 Missed Deadlines: 0
Total CPU Utilization: 95.0000%
RMS Schedulability Bound: 77.9763%
Status: Further Analysis Required (Bound < Utilization <= 100%)
rajababu@Rajas-MacBook-Air OS_LAB %
```

---

8. Observations

- MLFQ performs well for mixed workloads
- Lottery scheduling provides fairness but is random
- EDF is best for real-time systems but may fail if deadlines are tight
- RMS is good for real time monotonics

---

9. Advantages

- Better understanding of scheduling concepts
- Comparison between different algorithms
- Flexible design for adding more algorithms

---

10. Limitations

- Simplified simulation (no real OS kernel)
- No actual multiprocessing hardware usage
- Randomness in lottery scheduling may vary results

---

11. Future Improvements

- Add graphical visualization (charts/graphs)
- Implement more algorithms (RMS, Round Robin)
- Extend to real-time system simulation
- Use threads for better realism

---

12. Conclusion

This project helped us understand how different scheduling algorithms work and how they impact system performance. By comparing multiple algorithms, we learned that no single algorithm is best for all situations. The choice depends on system requirements such as fairness, responsiveness, and deadlines.

---

13. References

- Operating System Concepts – Silberschatz
- Lecture Notes

- Online resources

---

14. GitHub Repository

https://github.com/UmarSk2/G28_Project2_2